

Final Report

1. Introduction

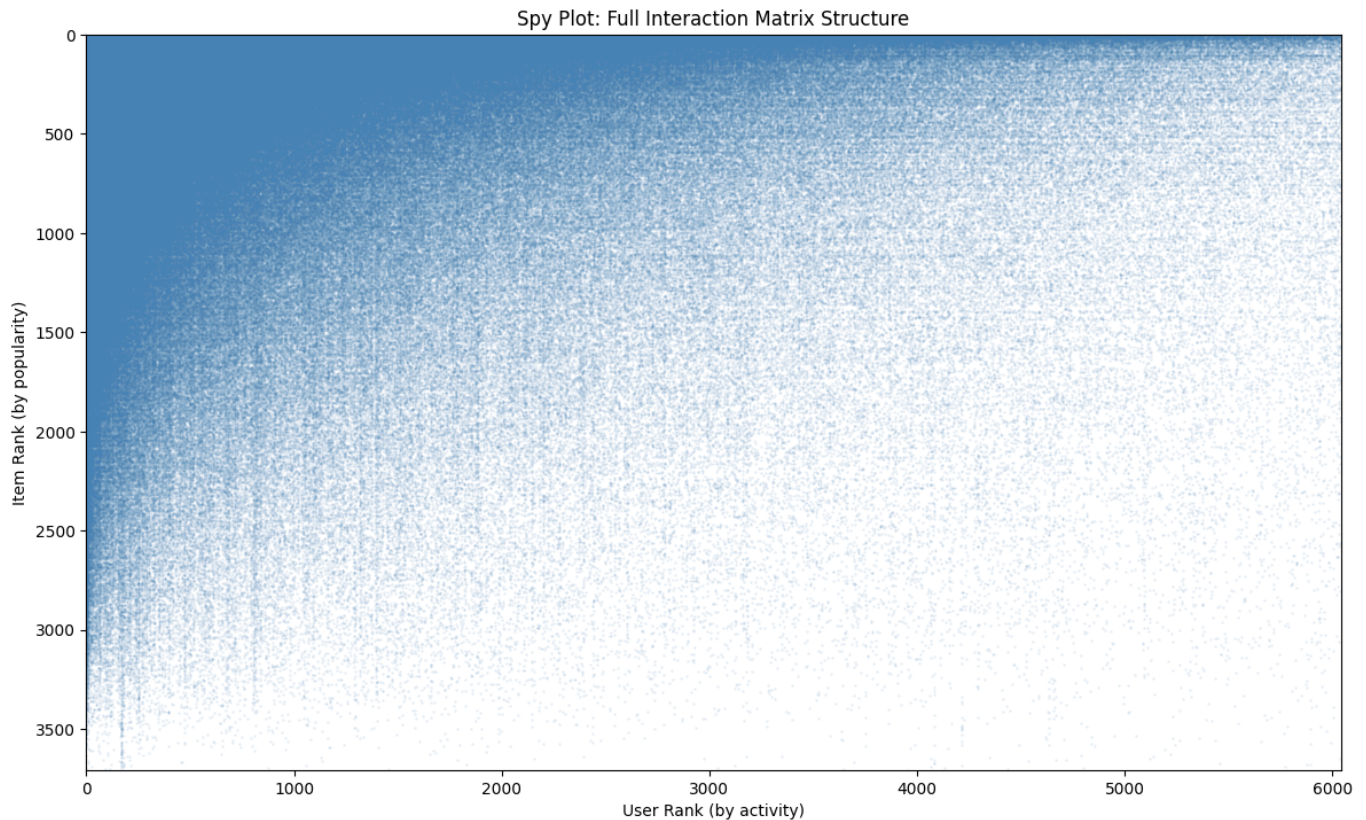
This project examines traditional recommender system methods using the MovieLens 1M dataset. This dataset is an industry standard and includes one million ratings from 6,040 users across 3,706 movies. We chose this dataset because it offers interaction timestamps needed for time-based evaluation, genre metadata useful for content-based methods, and enough scale to test different algorithm choices. Its common use in research allows us to compare our results with established benchmarks. The main question guiding this work is how similarity-based methods stack up against matrix factorization when evaluated under realistic conditions that consider temporal order.

The report is organized as follows. First, we summarize important insights from exploratory data analysis that inform modeling decisions. Next, we describe the offline evaluation framework, including the temporal split strategy and metrics. The following sections detail the implementations of content-based filtering, collaborative filtering, FunkSVD, and Alternating Least Squares, along with experimental results. The report ends with a comparative analysis, deployment recommendations, and noted limitations. Throughout, we focus on ranking quality, measured by nDCG@10, as the main optimization target.

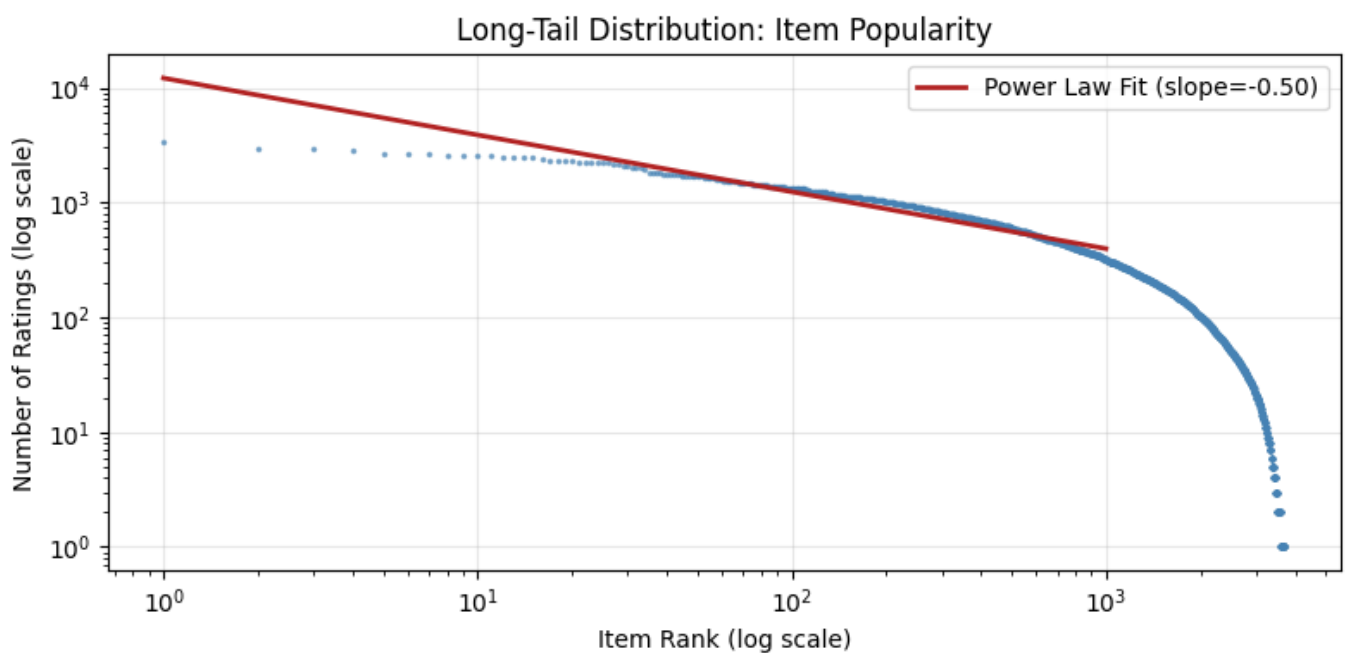
2. Exploratory Data Analysis

- Detailed report: [reports/1-exploratory-data-analysis.md](#)
- Notebook: [experiments/exploratory-data-analysis.ipynb](#)

The most remarkable feature of this dataset is its sparsity. Only 4.47% of user-item pairs have ratings, leaving 95.5% of the interaction matrix empty. This sparsity directly affects model selection. When we looked at user-pair overlap, the median number of co-rated items was only 9. Such low overlap makes memory-based collaborative filtering unreliable. Similarity estimates based on fewer than 10 shared items contain a lot of noise. Singular value decomposition of the rating matrix showed no clear elbow point, with over 50 components needed to capture even 30% of the variance. This high-dimensional preference structure indicates that users have truly diverse tastes that cannot be simplified to just a few latent factors.



Two data issues need to be addressed. First, item popularity follows a power-law distribution with a slope of -1.48 on a log-log scale. The top 20 movies make up about 5% of all ratings, even though they represent less than 1% of the catalog. This concentration risks naive models focusing on popularity instead of learning personalization. Second, 12% of items have fewer than 10 ratings, putting them in a cold-start situation where collaborative signals are not enough. Genre metadata serves as a backup for these items, but the 18-genre taxonomy only provides rough similarity.



The time structure of the dataset allows for evaluation based on time. The average rating stayed steady at 3.58 over the 2.8-year collection period, showing no noticeable changes. However, the time split reveals a significant cold start during evaluation: 60% of users in the validation set and 38% in the test set were not part

of the training. This situation is realistic, but it requires that models offer reasonable recommendations for users with no history, either through content-based alternatives or popularity-based defaults.

3. Offline Evaluation Strategy

- Detailed report: [reports/2-offline-evaluation-strategy.md](#)
- Notebook: [experiments/metrics-example.ipynb](#)

We use a chronological split with a 70/15/15 ratio for training, validation, and test sets. Ratings are arranged by timestamp. The earliest 70% makes up the training set, the next 15% is the validation set, and the final 15% is the test set. This method avoids data leakage by ensuring that models do not see future interactions during training. A random split would let the model access ratings that come after those it needs to predict, which would falsely boost performance estimates. The cold-start exposure mentioned earlier is a real challenge that systems in the real world must manage.

The main evaluation task focuses on ranking, not predicting ratings. Users engage with ordered recommendation lists instead of numerical scores, so the quality of the ranking gives a better idea of practical usefulness. We use nDCG@10 as the primary metric because it gives more importance to relevant items based on their position in the ranking, rewards hits that appear earlier, and provides scores between 0 and 1 that can be compared across users with different numbers of relevant items. The table below outlines the complete metrics framework.

Metric	Category	Purpose
nDCG@10	Ranking (Primary)	Position-weighted relevance normalized to [0,1]; primary optimization target
MAP@10	Ranking	Average precision emphasizing early hits
Precision@10	Ranking	Fraction of top-10 recommendations that are relevant
Recall@10	Ranking	Fraction of relevant items appearing in top-10
RMSE	Rating	Root mean squared error for rating calibration diagnostics
MAE	Rating	Mean absolute error, more robust to outliers
Coverage	Beyond-accuracy	Fraction of catalog recommended; detects popularity bias
Novelty	Beyond-accuracy	Measures recommendation of less popular items

All metrics are computed using the Microsoft recommenders library version 1.2.1 to ensure reproducibility and consistency with published benchmarks.

This evaluation framework measures a model's ability to rank relevant items based on historical interaction patterns. However, it does not address several important aspects of production systems. Online feedback loops, where recommendations affect later user behavior, are not captured by offline metrics. Long-term user satisfaction may differ from short-term ranking accuracy, especially if users prioritize surprise or variety over time. Preferences that depend on context, such as time of day, mood, or social situation, cannot be evaluated

using this dataset. Therefore, offline evaluation is necessary for filtering candidate approaches, but it is not enough for making final deployment decisions.

4. Similarity-Based Recommenders

- Detailed report: [reports/3-similarity-based-recommenders.md](#)
- Notebook: [experiments/similarity-based-recommenders.ipynb](#)

We implemented and compared three classes of similarity-based models: content-based filtering, collaborative filtering, and a hybrid approach.

Content-based filtering uses item metadata. The baseline model used Jaccard similarity on binary genre sets. An improved version utilized TF-IDF on a combined "metadata soup" of movie titles and genres to capture more specific similarities like sequels.

Collaborative filtering employed item-item similarity based on user interaction vectors. We compared cosine similarity against Pearson correlation, which centers data to account for user rating bias.

The hybrid model combined the scores of the best CF and CB models using a weighted sum, aiming to mitigate the sparsity of collaborative signals with content metadata.

4.1 Results

The table below summarizes the performance of the models at K=10 on the test set.

Model	NDCG@10	Precision@10	Recall@10
Hybrid (CF + TF-IDF, alpha=0.8)	0.1493	0.1360	0.0861
CF Item-Item (Cosine)	0.1488	0.1380	0.0865
CF Item-Item (Pearson)	0.1291	0.1160	0.0609
CB (Jaccard)	0.0339	0.0320	0.0131
CB (TF-IDF)	0.0306	0.0290	0.0170

4.2 Discussion

Collaborative filtering with cosine similarity proved to be the most effective approach, significantly outperforming content-based methods. This confirms that for the relatively dense MovieLens dataset, the patterns of collective user behavior are far more predictive of preference than static metadata like genres.

Interesting nuances appeared in the comparison. Pearson correlation underperformed compared to simple cosine similarity. While Pearson theoretically corrects for user bias, in practice, the mean-centering operation on sparse vectors likely introduced noise or dampened valid signals. While Jaccard on genres yielded better top-ranking precision, TF-IDF on titles improved recall. This indicates that including titles helps the model find specific relevant items like sequels that genre-matching misses, even if it ranks them lower on average.

The weighted hybrid model showed performance comparable to the pure CF baseline, achieving a slightly higher NDCG (0.1493 vs 0.1488) but slightly lower recall. This suggests that blending content signals can

marginally improve ranking order by refining the score ties, even if it doesn't significantly expand the set of retrieved items. However, the gain is minimal, reinforcing that collaborative signals are the primary driver of quality here.

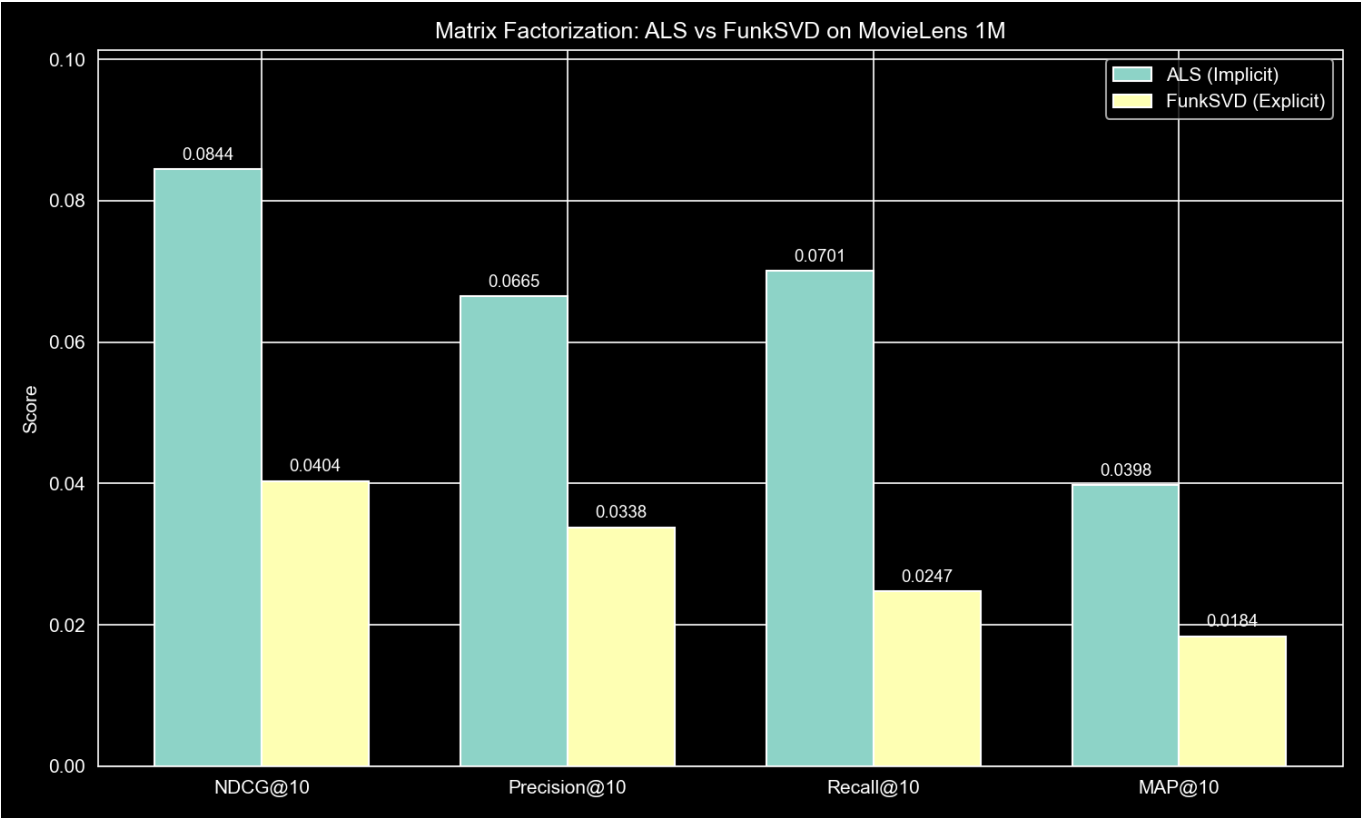
5. Matrix Factorization

We implemented two matrix factorization approaches: ALS (Alternating Least Squares) with implicit feedback and FunkSVD with explicit ratings.

5.1 Results

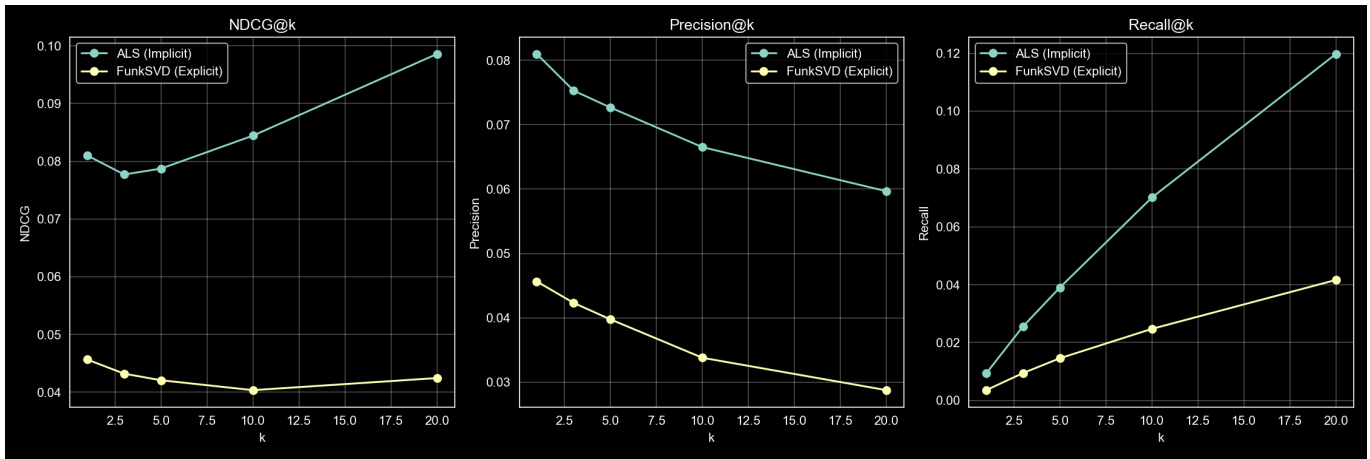
Model	NDCG@10	Precision@10	Recall@10	RMSE
ALS (Implicit)	0.0844	0.0665	0.0701	N/A
FunkSVD (Explicit)	0.0404	0.0338	0.0247	0.886

ALS outperforms FunkSVD on all ranking metrics by roughly 2x.



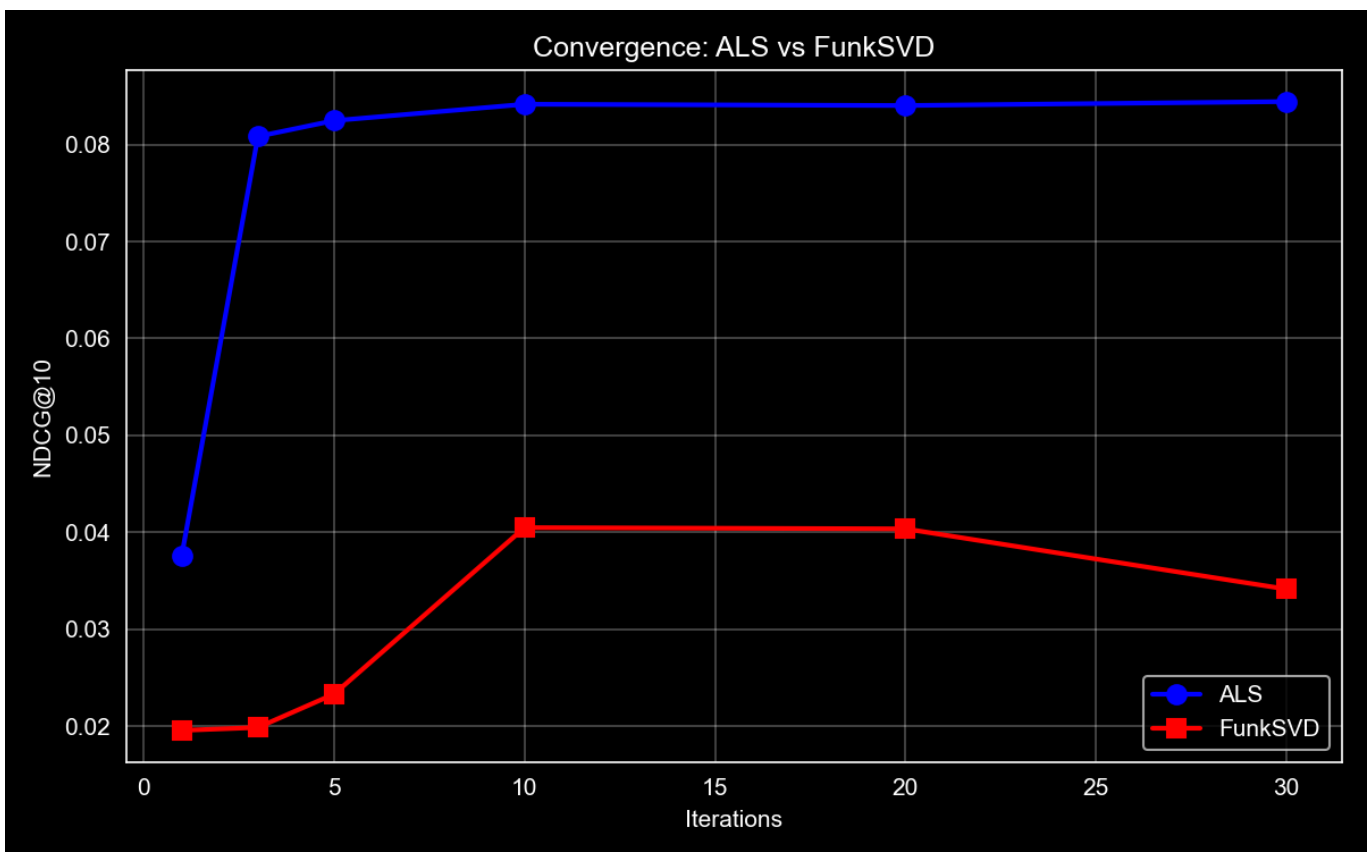
5.2 Metrics vs K

ALS beats FunkSVD at every k value. Precision drops as k grows (more recommendations = harder to stay precise), while recall increases (more chances to hit relevant items). This is the standard precision-recall tradeoff. NDCG accounts for ranking position, so it stays more stable across different k values.



5.3 Convergence Analysis

ALS converges quickly and stays stable - the closed-form solution with regularization keeps it from overfitting. FunkSVD peaks around 10-20 iterations then drops, a sign of overfitting: it keeps minimizing training RMSE but that hurts ranking quality on test data. This shows why early stopping matters for SGD methods, and why optimizing for rating prediction doesn't directly translate to good recommendations.



5.4 Discussion

ALS (implicit) outperforms FunkSVD on ranking metrics because it's optimized for the ranking task directly. FunkSVD tries to predict exact rating values which doesn't necessarily translate to good rankings. ALS treats ratings ≥ 4 as positive signals and learns what users prefer, while FunkSVD treats a 3-star and 5-star rating very differently even though both might indicate interest.

The choice of collaborative filter should account for these differences. One crucial operational distinction is that ALS is parallelizable and can scale across multiple compute nodes, while FunkSVD's sequential gradient

updates do not parallelize as naturally.

6. Summary and Analysis

This section brings together results from all experiments and addresses key questions about model selection, failure modes, and deployment strategy.

6.1 Performance Comparison

The table below consolidates results from all implemented models, ordered by their primary metric nDCG@10.

Rank	Model	NDCG@10	Precision@10	Recall@10
1	Hybrid (CF + TF-IDF, $\alpha=0.8$)	0.1493	0.1360	0.0861
2	CF Item-Item (Cosine)	0.1488	0.1380	0.0865
3	CF Item-Item (Pearson)	0.1291	0.1160	0.0609
4	ALS (Implicit)	0.0844	0.0665	0.0701
5	FunkSVD (Explicit)	0.0404	0.0338	0.0247
6	CB (Jaccard)	0.0339	0.0320	0.0131
7	CB (TF-IDF)	0.0306	0.0290	0.0170

Item-item collaborative filtering with cosine similarity is the most effective method. The hybrid model shows only a slight improvement over pure CF, with scores of 0.1493 and 0.1488. This small gain does not justify the added complexity. Content-based methods perform the worst overall, which shows that genre metadata only offers a rough differentiation for this dataset. Matrix factorization methods do not perform as well as memory-based CF in this case, which goes against what the sparsity analysis suggested. The likely reason is that the item-item similarity matrices in MovieLens 1M are still dense enough, with 3,706 items by 3,706 items, to provide reliable neighborhood estimates. Meanwhile, the latent factor models struggle to do better than direct similarity computation in a dataset of this size.

6.2 Failure Modes

Each model class has distinct failure patterns that arise under certain conditions.

Content-based filtering struggles when metadata is not enough to differentiate items. Two drama films from different decades with completely different audiences may seem alike based solely on genre. The 18-genre taxonomy is too broad to capture subtleties in tone, style, or target demographic. This approach also cannot make use of the behavioral signals that show hidden links between items, like users who enjoy both horror films and true crime documentaries.

Collaborative filtering has problems with new users and new items. With 60% of validation users missing from training data and 12% of items receiving fewer than 10 ratings, collaborative filtering cannot offer useful recommendations in these situations. The item-item approach helps somewhat with new users since they can

receive suggestions after rating just a few items, but it still struggles with new items because they have no interaction history to rely on.

FunkSVD suffers from overfitting. The training RMSE keeps dropping even after ranking performance reaches its peak. This shows that the model memorizes training patterns instead of learning general preferences. Early stopping is important, but even with careful tuning, FunkSVD's focus on explicit rating optimization doesn't translate effectively into ranking quality. ALS performs better by treating the problem as implicit feedback, but both matrix factorization methods do not perform as well as direct similarity computation on this dataset.

6.3 Bias Analysis

Three forms of bias affect model training and evaluation.

Popularity bias comes from the power-law distribution of item ratings. The top 20 movies make up 5% of all interactions, even though they represent less than 1% of the catalog. Models trained on this data tend to favor popular items because these dominate the training signal. A model that recommends only the most popular unwatched items can achieve significant accuracy without any personalization. The evaluation framework does not explicitly penalize popularity bias, so reported metrics may exaggerate the level of true personalization.

Activity bias results from the long tail of user engagement. Power users who rate hundreds of movies contribute a lot to the training data. If these users have unusual preferences, the model may pick up patterns that do not apply well to casual users with fewer ratings. The median user has rated 96 movies, while the mean is 166, influenced by highly active participants.

Cold-start exposure affects evaluation directly. The temporal split creates realistic cold-start conditions: 60% of validation users and 38% of test users never showed up in training. This means that overall metrics mix warm-user and cold-user performance. Models with strong fallback strategies may seem better than those designed only for warm users. This is fine for production systems, but it can hide performance differences within the warm-user group.

6.4 Deployment Recommendation

For this dataset and use case, we suggest using item-item collaborative filtering with cosine similarity as the main recommendation engine. This recommendation is based on four factors.

First, it shows the best ranking performance among all tested methods. The nDCG@10 score of 0.1488 is almost equal to that of the hybrid model while being easier to implement and maintain.

Second, item-item similarity matrices are easy to understand. When a recommendation doesn't work or seems odd, developers can check which items in the user's history influenced the suggestion. This clarity helps with debugging and builds trust with stakeholders.

Third, the computational needs are reasonable. The item-item similarity matrix measures 3,706 by 3,706, which fits well in memory. By precomputing and caching this matrix, we can provide real-time recommendations with low delays.

Fourth, cold-start solutions are easy to set up. For new users with no history, popularity-based recommendations are a solid default. For new items, content-based similarity using genre metadata can kickstart recommendations until there are enough interactions.

When deploying in production, include genre-based content filtering for cold items, popularity-based recommendations for completely new users, and monitor recommendation diversity to catch any excessive popularity bias.

6.5 Limitations and Next Steps

This project focused on classical methods and offline evaluation, which leaves several directions unexplored.

Deep learning techniques such as neural collaborative filtering, autoencoders, and sequence models have achieved strong results on similar datasets. These methods can capture non-linear interaction patterns that linear matrix factorization misses. However, they need more careful tuning and larger computing resources.

Hybrid neural models that combine collaborative signals with content features in a learned way may perform better than the simple weighted hybrid tested here. Approaches like wide-and-deep networks or two-tower architectures could better use the available metadata.

Online evaluation through A/B testing would show whether the offline improvements lead to increased user engagement in real situations. Offline metrics cannot capture feedback loops, novelty preferences, or long-term user satisfaction that only appear after deployment.

Contextual features like time of day, device type, or session context are missing from MovieLens but exist in actual systems. Adding these signals could improve recommendation relevance for specific moments.

The evaluation framework would benefit from reporting based on user activity level and item popularity tier. This would clarify whether models really personalize or just exploit popularity.